

Walkthrough

A quick tour of how I built this and why I made the calls I did — not a restatement of the assignment, just the reasoning behind the pieces that weren't obvious.

What it does

Type 20 randomly generated letters as fast and accurately as you can. Each wrong keypress adds a 0.5s penalty to your time. Beat your own previous best and you get a Success screen; otherwise it's Failure/Try Again. There's a leaderboard of everyone's best times, and your own best time is remembered locally even before you've registered an account.

Stack, and why

Bun + TypeScript + GraphQL Yoga + Prisma + PostgreSQL on the backend, React + Vite + TypeScript + Tailwind on the frontend, the whole thing containerized with Docker Compose for local dev. This was largely a fixed requirement for the assignment, but a few sub-decisions inside it were mine to make.

Stateless JWT over refresh tokens. I initially considered a refresh-token setup, but talked myself out of it: this app has no session data worth revoking mid-flight, no admin panel, nothing that needs instant logout-everywhere. A single signed JWT with a 1-day expiry, sent as a Bearer token on every request, does everything this app actually needs without the extra moving parts (a token store, rotation logic, revocation lists) that a refresh-token system would drag in. It's a tradeoff I'd revisit if this ever needed "log out this device remotely" — it currently can't do that.

Denormalized bestTimeMs on the User row, updated atomically. Every completed game gets its own permanent `GameResult` row (so history and leaderboard ordering by anything other than "current best" both stay possible), but I also copy the winning time onto `User.bestTimeMs` directly, because the leaderboard query runs on every page load and I didn't want it computing `MIN(timeMs)` across the whole `GameResult` table each time. The risk with denormalizing like this is a race: two submissions landing close together could both read "no existing best" and both try to write, or a slower one could overwrite a faster one that was saved microseconds earlier. I handle that with a single `$transaction` that always inserts the `GameResult` row, then does a conditional `updateMany` — `WHERE bestTimeMs IS NULL OR bestTimeMs > $newTime` — so the update only ever takes effect if the new time is genuinely still better at the moment it's applied, rather than trusting whatever the server read a few milliseconds earlier. I have integration tests against a real Postgres instance covering this specifically: a faster time always wins, a slower time never overwrites, and every attempt (win or not) still gets its permanent row.

GraphQL errors carry a machine-readable code in extensions. Rather than throwing plain error strings, every error the API returns includes an `extensions.code` like `EMAIL_TAKEN`, `INVALID_CREDENTIALS`, `UNAUTHENTICATED`, or `BAD_USER_INPUT`. That's what lets the frontend react differently to different failures instead of pattern-matching on error text, which breaks the moment a message gets reworded.

Login failures are indistinguishable on purpose. Whether the email doesn't exist or the password is wrong, `login` returns the exact same "Invalid email or password" error either way. Splitting those into two different messages would let anyone probe which emails are registered on the app at all — a real, well-known weakness (user enumeration), not a hypothetical one.

gameHistory and submitGameResult both scope entirely off the caller's JWT, never a client-supplied ID. Neither takes a `userId` argument. `submitGameResult` writes to `context.userId`, and `gameHistory` filters `WHERE userId: context.userId` — both taken from the verified token, not anything the request body could claim. That's a deliberate choice: it means there's no argument to validate or forget to check, because there's no argument at all. One user reading or writing another user's data isn't a permission check that could have a bug in it; it's a request shape that doesn't exist.

Frontend state

I used React Context (`AuthContext`, `ThemeContext`) rather than pulling in a state library — the app only has two genuinely global pieces of state (who’s logged in, light or dark mode), and Context covers that without adding a dependency for a problem this small.

Inside the game timer specifically, I split state deliberately: the running penalty total and the game’s start timestamp live in `useRef`, not `useState`, because they get read and written on every keystroke and don’t need to trigger a re-render themselves — only the *displayed* elapsed time (ticked by a `setInterval`) is `useState`, since that’s the one value that actually needs to repaint the screen.

Design system

Tailwind v4, with a custom `ruby-*` color scale (50 through 950) derived from a single base color (`#E0115F`) and defined once via an `@theme` block, paired with white/neutral surfaces. Both light and dark mode are supported, toggled manually rather than only following the OS preference, with the choice persisted in `localStorage`.

Testing approach

Integration tests run against a real PostgreSQL instance rather than a mocked Prisma client — I’d rather catch a real SQL/transaction bug than get a false sense of safety from a mock that doesn’t actually enforce the database’s own guarantees. The tradeoff is these tests need a running db container, which is a fair price for testing the one piece of logic (the atomic best-time update) where a bug would be genuinely hard to notice by just clicking around the UI.

Deployment

The live version runs on three separate free-tier services, deliberately kept apart rather than crammed onto one host: Neon for Postgres (its free tier doesn’t expire, unlike most managed-Postgres free tiers, which matters if I want this link to still work in six months), Render for the backend (it builds straight from the same Dockerfile I use locally — zero deployment-specific backend code), and Vercel for the static frontend build. The only code change deployment required was making the frontend’s GraphQL endpoint URL read from an environment variable instead of being hardcoded to `localhost`, so the same codebase runs unmodified in both environments.

What I’d flag as known limitations

Client-reported `timeMs` is trusted at face value beyond basic sanity validation — a determined user could submit a fabricated time, since there’s no server-side timing verification. I considered this out of scope for the assignment’s size, but it’s the honest gap if this were a real competitive product. Similarly, there’s no password reset flow and no refresh-token rotation — a single JWT with a fixed expiry was the right amount of auth surface area for what this app actually needs, not a shortcut I’d defend at a larger scale.